

Triangular mesh for Eirene: how to prepare and use it.

A.S.Kukushkin, Nov. 07, 2005

A triangular mesh is necessary for the non-linear version of the Eirene code [1] which treats neutral-neutral collisions also outside the regular grid used for the plasma components. This brief instruction describes how to produce such a mesh and make it usable in the context of the B2-Eirene package, version solps4.2 [2], using standard solps tools such as DG and UINP together with a triangular mesh generator TRIA [3] also integrated in the solps structure.

1. File arrangements

Every triangular mesh is represented with three separate files containing the data on the mesh nodes, on connection of these nodes, and on the cell neighbourhood. These files are located in the `$SOLPSTOP/data/TRIANG/$DEVICE` directory and their names are

`$DEVICE.tria.<ID>.nodes`,

`$DEVICE.tria.<ID>.cells`, and

`$DEVICE.tria.<ID>.links`

(here `<ID>` stands for the mesh identifier, e.g., 007 for the mesh #007). These files should be linked to the files `fort.33`, `fort.34`, and `fort.35`, respectively, in the directory where the Eirene runs.

The neutral background data calculated in Eirene and used for the neutral-neutral interactions are saved on the file `fort.13` which should therefore be kept for a smooth continuation of the run.

2. General sequence of actions

In order to prepare a B2-Eirene run with neutral-neutral collisions, using solps4.2 package, the following steps are needed.

(a) A converged B2-Eirene run in the same geometry but without neutral-neutral collisions should be selected as the starting point, see an example in Fig. 1, and the working directory containing the usual files necessary to launch the run should be prepared.

(b) The input data for the triangular mesh should be prepared using the DG interface, and the DG output should be produced and linked to the standard place with the `lns` command.

(c) The `uinp` command should be run in the working directory to prepare the input for a special run of Eirene stand-alone, which would produce the input file for the TRIA triangular mesh generator, combining the external geometry specified with DG with the regular grid geometry taken from the `fort.30` file.

(d) The TRIA mesh generator should be run to produce the triangular mesh outside the regular B2 grid, Fig. 2.

(e) The TRIAGEOM program should be run to produce the combined triangular mesh covering the whole computational domain, Fig. 3. The B2 data on the plasma parameters will then be translated to this grid before, and the neutral data calculated in Eirene will be translated to the regular B2 grid after every call to Eirene.

(f) The mesh files should be stored and linked according the conventions described in the previous section.

(g) The mesh files should be converted in the DG template format. This template can then be imported in DG for viewing.

(h) The Eirene input file should be corrected manually – the `uinp` command is still not fully up-to-date.

Following steps (c) through (g) can be assisted by a `triang` script.

3. Data preparation

The DG interface should be used to prepare the input for TRIA and to visualize the result.

(1) The usual procedure of creation of a model including a valid B2 grid should be done. An example of one ASDEX Upgrade model is shown in Fig. 1.

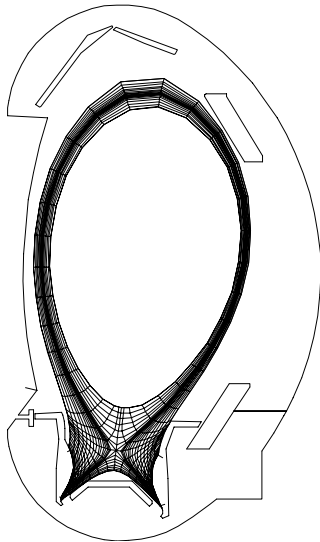


Fig. 1. An example DG model for ASDEX Upgrade

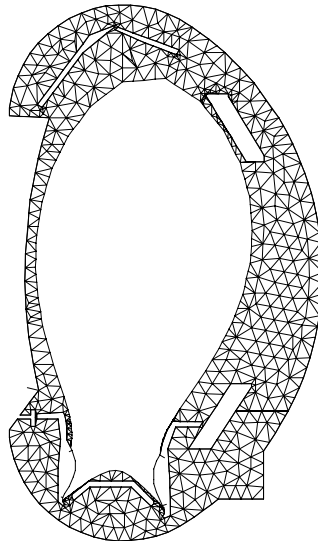


Fig. 2. External triangular mesh produced with the TRIA code

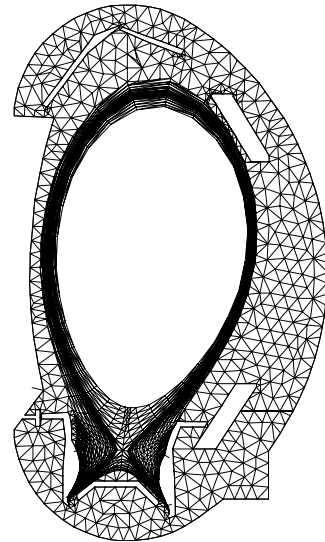


Fig. 3. Combined triangular mesh produced with the TRIAGEOM code

(2) Add a variable “TRIA-EIRENE parameters” (Variables|Add menu). Specify the desirable size of the triangles for the whole mesh. If you want to have finer mesh in some areas, add variable “Mesh Refinement Zone” and specify the zone and the desirable mesh resolution there. There can be several occurrences of this variable, each determining a separate mesh refinement zone. Note that the mesh refinement zone in TRIA can only be specified as a rectangle with sides oriented along the co-ordinate axes. In order to approximate other shape of the mesh refinement zone, several such rectangles have to be used. In case of overlapping rectangles, the minimum cell size is used for the intersection.

(3) Specify the mesh-bounding contours via the “TRIA-EIRENE parameters” variable. In order to do this, assign a certain value of “Index” to every element in the DG model. Zero value means that this element is not involved in the mesh specification. All the elements having the same non-zero value of “Index” form the contour which must be continuous. Keep in mind that these contours will be complemented with the boundaries of the regular grid connected to the elements marked as the “target edges” in the “Target specification” dialogs. Therefore, the contours you select must be either closed, or ending with the “target edges”. The index values are negative for the contours (polygons) confining the mesh, and positive for the contours limiting regions to be excluded from the mesh. The absolute values of the

index must start from 1 and increase without gaps (1, 2, 3, ...). This procedure is illustrated in Figs. 4 – 9.

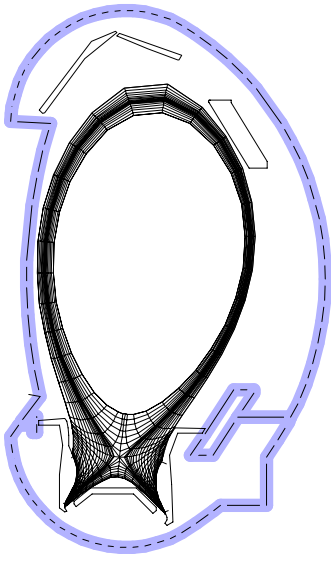


Fig. 4. Elements with index of -1 selected

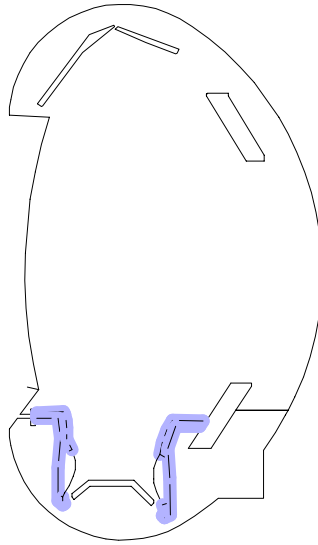


Fig. 5. Elements with index of 2 selected

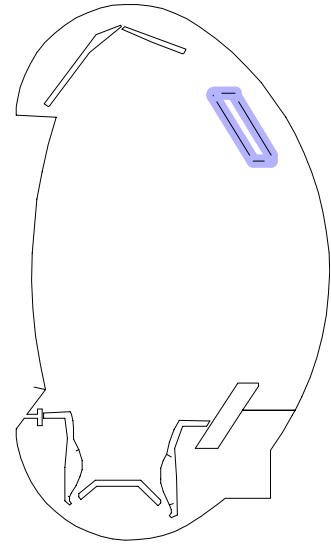


Fig. 6. Elements with index of 3 selected

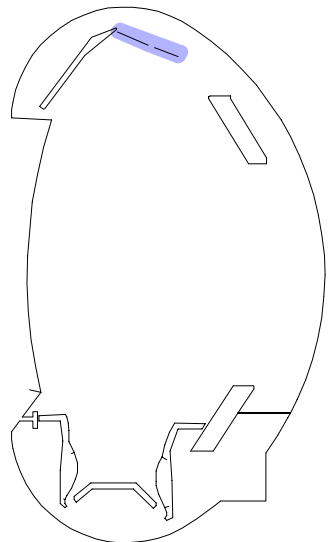


Fig. 7. Elements with index of 4 selected

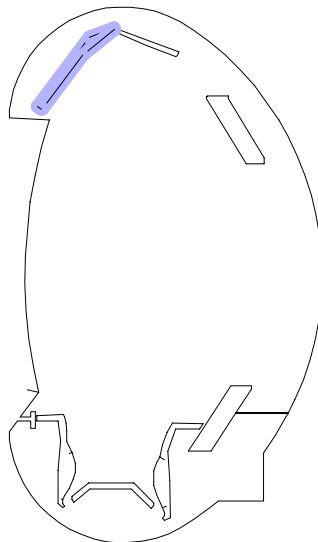


Fig. 8. Elements with index of 5 selected

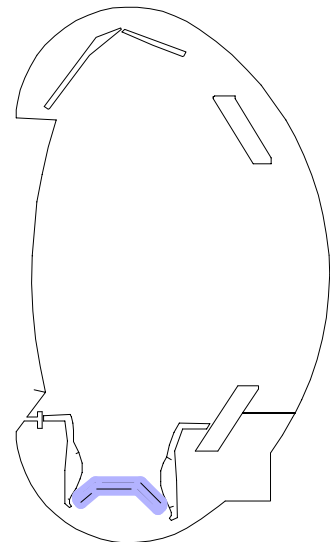


Fig. 9. Elements with index of 6 selected

Another example, for ITER geometry, is shown in Figs. 10–13. The resulting mesh consists of two disconnected parts, Fig. 13.

(4) Produce the DG output (File|Output menu) and run the `lns` command with the DG model filename without extension, as usual.

4. Producing the triangular mesh

In the working directory where the run will eventually start, type `triang` to call the script which will guide you through the steps (c) to (g) from the section 2. At every step, the script prints the list of actions from which you can select the next one entering the letter capitalized in the list. If the default action shown in brackets suits you, just press <Enter>. The expanded

action list with some explanation can be displayed by entering ‘h’ (Help action). Following the lines of section 2, you should first select ‘U’ to link the DG output to the current directory and to run `uinp`, then ‘e’ to run Eirene, then ‘t’ to run TRIA, then ‘g’ to run TRIAGEOM, then ‘s’ to assign the identifier, save the mesh, and link the corresponding files to `fort.3[345]` in the current directory, then ‘c’ to convert the mesh in a DG template (and, probably ‘C’ to convert also the combined mesh). After this, import the corresponding template in DG (File|Import|Template menu) to see the produced mesh.

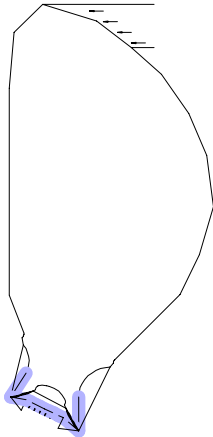


Fig. 10. Elements with index of -1 selected

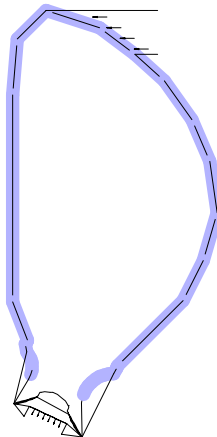


Fig. 11. Elements with index of -2 selected

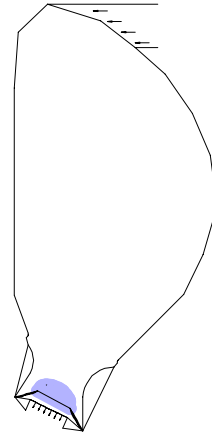


Fig. 12. Elements with index of 3 selected

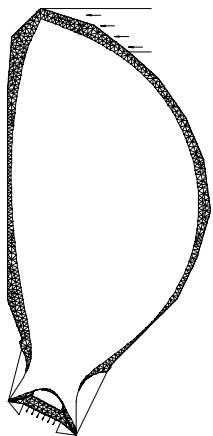


Fig. 13. External triangular mesh produced with the TRIA code

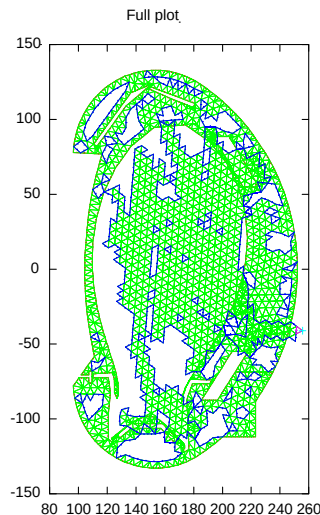


Fig. 14. Example of “plot on error”: the state when TRIA just noticed doing wrong.

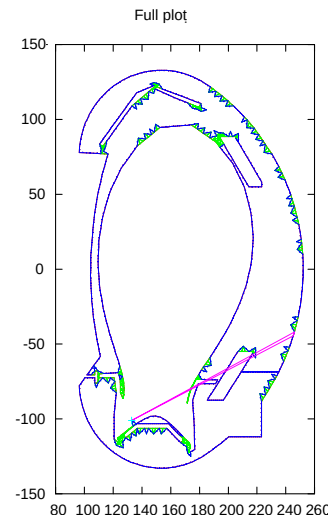


Fig. 15. The same after a trap at the step 281. The first wrong step is clearly seen.

For most of the actions, the upper case and lower case responses are equal (e.g., both ‘T’ and ‘t’ mean ‘call TRIA’). There are three exceptions:

‘U’ links the DG output to the current directory before running `uinp`, whereas ‘u’ merely runs `uinp`. This is convenient when you return to an old case, where the proper links exist and the “standard place” links point already to another model output (i.e., the `lms` command was run with another filename).

‘C’ converts to a DG template the combined mesh produced with the ‘g’ action, whereas ‘c’ does this for the original triangular mesh produced with the ‘t’ action. In the first case,

the resulting template is called '`%g<ID>.tria`', and in the second case it is '`%a<ID>.tria`' – therefore, you can keep them both, the names are different. Since the identifier is assigned to a mesh when it is being stored, the “s” (Store) action should precede the conversion – otherwise, `%g%` and `%a%` are used. The templates are created in the `$SOLPSTOP/src/DivGeo/dg/class/$DEV/templates` directory, if it exists, or in `$SOLPSTOP/src/DivGeo/dg/class/$DEV` otherwise. The `DEV` environmental variable is determined from the directory where the DG output files are located (and where the links in the current directory are pointing) or equal to `$DEVICE` if the DG output files were copied, not linked, to the current directory.

'V' requests viewing the graphical output from TRIAGEOM, and 'v' from TRIA. Note that these actions require that the TRIA and TRIAGEOM have been compiled with the graphics package, which is not always the case. Usage of DG for viewing the meshes is therefore recommended.

Two other useful actions are 'l' and 'r'. The first outputs a list of meshes stored in the TRIANG mesh archive, and the second allows you to cleanly remove all the files related to the specified meshes, including the converted DG templates.

It was mentioned above that the `fort.30` file is required to produce the input file for TRIA. The two actions, B2 and reMap, were originally foreseen to assist in creating this file. However, they are not realized yet and it is your responsibility to ensure the presence of the correct `fort.30` file in the working directory.

Finally, there are two actions designed to help localizing possible errors occurring during the TRIA call (mostly, errors in the input data for TRIA).

'P' or 'p' produces the plots of the geometry structures as input by TRIA and of the current state of the TRIA operation. This action should be called in the case of an error message from TRIA. An example of such a “plot on error” is given in Fig. 14 (a self-intersecting contour, an older version of TRIA without check for such self-intersections; the current version would display an error message showing the problem).

'A' or 'a' sets a trap in TRIA which then stops after the specified number of iterations. This can help to localize the original problem, see Fig. 15.

5. Correcting the Eirene input files

Since the `uinp` command is not yet fully up-to-date, some manual corrections to the Eirene input files are necessary. Whereas the geometry data specified in DG are correctly transferred to the Eirene input files (`input.eir` and `test.eir`), the data on particle interactions are not. The following corrections are therefore to be done:

- (a) blocks 4 and 5 should be taken from an input file where the reactions and species were set for this Eirene version.
- (b) In block 6, the reference to the TRIM model should be set manually.
- (c) In block 6, the “surface specification blocks” (five lines each, starting with `SURFMOD` in the first line) must be corrected if sputtering is specified in the model. First, the 3rd and 4th integers in the second line should be set equal to the position of the particles to be released by, respectively, physical and chemical sputtering in the particle list, and the value of the chemical sputtering yield should be introduced as the second real value in the last line of the block.

- (d) In the block 7, the minimum number of histories (the last integer in the second row for each stratum) should be specified. You can do it through DG for the targets and gas puffing strata.
- (e) In block 14, data of the neutral background species should be added before the line of two integers specifying the B2 grid dimensions. The easiest way is to take these lines from another file set up for the same particle composition (the lines with two integers, the second being -13).

References

- [1] D. Reiter, <http://www.eirene.de/>
- [2] A.S. Kukushkin, H.D. Pacher, V. Kotov, et al., Nucl. Fusion **45** (2005) 608.
- [3] P. Börner, B. Küppers, private communication.